תיאום MEP בזמן אמת — מה נעשה, בשפה פשוטה

תקציר בעברית של תרגיל הבית. הפתרון המלא, עם הפסאודו-קוד, התרשימים וה-JSON, נמצא במסמך באנגלית ובריפו.

Elnatan Davidovitch
elnatan1990@gmail.com · 050-6998223 · linkedin.com/in/elndav
code: github.com/elnatan1990/maof · live: maof-mep.vercel.app

התרגיל עוסק בתיאום MEP (מערכות מיזוג, חשמל ואינסטלציה) וביקש שלושה דברים: איך מחשבים נתיב שעוקף מכשול בתלת-ממד, איך המידע זורם בזמן אמת כשמישהו מזיז קורה ב-Revit (תוכנת תכנון הבניין בתלת-ממד של Autodesk), ואיך משלבים AI בלי שהוא יפר כללים הנדסיים. הנה מה שבניתי, איך זה עובד, ואיפה רואים את זה בפועל.

מה מצורף ואיפה מסתכלים.

- **המסמך באנגלית** (2 עמודים + נספח עם הקוד) — התשובה המלאה לשלושת החלקים.
- **דף אינטראקטיבי** maof-mep.vercel.app — סצנה תלת-ממדית: מזיזים את הקורה בסליידר ורואים את התעלה מנותבת מחדש, את מסלול המידע בין המערכות ואת שער הבטיחות (הבדיקה שכל הצעה של AI עוברת).
- **הקוד, הטסטים וסכמות ה-API** github.com/elnatan1990/maof — סקריפט Python קצר (`python -m routing`) שמדפיס את כל המספרים שבמסמך, ו-22 טסטים שמוודאים אותם.

1 הנתיב מסביב לקורה

הבעיה בשפה פשוטה. תעלה ישרה מנקודה A לנקודה B חותכת קורת בטון. צריך לעקוף את הקורה, לשמור ממנה מרחק של לפחות 15 ס"מ, ולעשות זאת עם כמה שפחות פניות, בזוויות נקיות של 45° או 90°.

מה עשיתי. במקום לחפש נתיב ברשת תלת-ממדית (איטי, וצריך "ליישר" את התוצאה אחר כך), מחשבים את העקיפה ישירות:

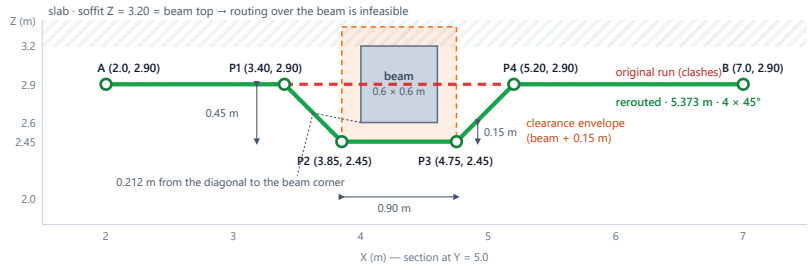
1. **מגדילים את התיבה של הקורה** ב-15 ס"מ לכל כיוון (ובתעלה אמיתית, גם בחצי הגובה של התעלה, כשעוקפים מלמטה). כל מסלול שלא נוגע בתיבה המוגדלת נמצא לפחות 15 ס"מ מהקורה.
2. **בודקים ארבע אפשרויות**: מתחת לקורה, מעל, משמאל, מימין. לכל אחת בודקים אם היא בכלל אפשרית. מעל הקורה אי אפשר, כי הקורה צמודה לתקרת הבטון. מתחת לקורה צריך שיישאר מקום מעל התקרה המונמכת (תקרת הגבס). מהצדדים צריך להישאר בתוך אזור העבודה, אבל הקורה חוצה אותו לכל רוחבו.
3. **בוחרים את הזולה**: פחות אורך נוסף ופחות פניות. כשיש תיקו, בוחרים תמיד באותו סדר, כך שאותה שאלה נותנת תמיד אותה תשובה. זה חשוב כדי שאפשר יהיה לעקוב ולבדוק אחר כך.
4. **אם אף אפשרות לא מתאימה,** המערכת לא ממציאה מסלול. היא מחזירה "אין מסלול" עם הסיבה (למשל: החלל מעל התקרה נמוך מדי) ופותחת משימת תיאום לקונסטרוקטור: שרוול דרך הקורה, הנמכה מקומית, או הזזת הקורה. כלי תיאום חייב להגיד למה אין פתרון.
5. **בודקים את התוצאה** עם אותם כללים שבודקים גם הצעות של ה-AI (חלק 3).

5.373 m
אורך המסלול החדש (במקום 5.0 מ', כלומר 37 ס"מ נוספים)

4 × 45°
ארבעה אביזרים, כולם בזווית נקייה

0.15 m
המרחק הקטן ביותר מהקורה, נמדד ולא מוערך

המסלול יורד ב-45° מתחת לקורה, עובר ישר מתחתיה ועולה חזרה: A → (3.40, 5, 2.90) → (3.85, 5, 2.45) → (4.75, 5, 2.45) → B (5.20, 5, 2.90). הגרסה עם פניות של 90° ארוכה יותר (5.90 מ'), אז 45° עדיף גם באורך וגם באיבוד הלחץ (מפל הלחץ). מסלול "V" עם שתי פניות בלבד נפסל: הרגליים שלו היו צריכות זווית של 11–14° (אין אביזר כזה) והיו עוברות 9 ס"מ בלבד מהקורה.



חתך במישור Y = 5: הקורה (אפור), התיבה המוגדלת, כלומר אזור המרווח של 15 ס"מ (כתום), התעלה המקורית (אדום מקווקו) והמסלול החדש (ירוק).

ומה קורה עם תעלה אמיתית? תעלה של 40×30 ס"מ (400×300 מ"מ) עם בידוד צריכה לרדת לגובה מרכז 2.26 מ', כלומר תחתיתה ב-2.07. אם התקרה המונמכת נמצאת ב-2.70 מ' (גובה מקובל בבניין משרדים), אין מקום, והמערכת תחזיר "אין מסלול" במקום להסתיר את הבעיה. בדף האינטראקטיבי אפשר לראות את זה: לבחור "400×300", להדליק את התקרה, ולהזיז אותה.

למה לא שיטות אחרות? חיפוש A* על רשת של קוביות קטנות (ווקסלים) איטי, דורש זיכרון רב ומייצר "מדרגות". גרף נראות (קווים ישרים בין פינות המכשולים) נותן זוויות שאין להן אביזר תעלה. RRT* (שיטה של דגימה אקראית, שנבנתה לזרועות רובוטיות) נותן כל פעם תוצאה אחרת ולא מתאים לתעלות. כשיש יותר ממכשול אחד בדרך, השלב הבא הוא גרף דליל על פאות התיבות עם חיפוש A* שמעניש פניות, ואז "חיתוך פינות" ל-45°. זה מתואר במסמך, לא נבנה בקוד, כי התרגיל ביקש גישה ולא מוצר.

2 מה קורה כשהאדריכל מזיז קורה ב-40 ס"מ

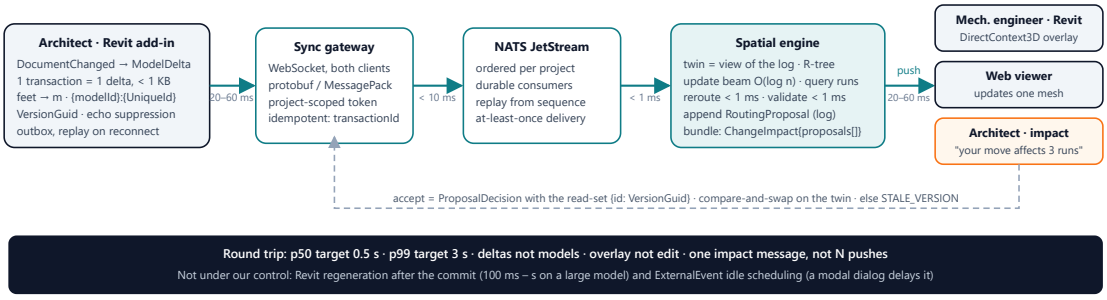
בשפה פשוטה. התוסף ב-Revit (תוכנה קטנה שרצה בתוך Revit) מזהה שהקורה זזה ושולח לענן רק את ההבדל: התיבה החדשה שמקיפה את הקורה (מיקום ומידות), המזהה שלה והגרסה שלה. זו הודעה של פחות מ-1 KB, לא מודל של מאות מגה-בייט. בענן נשמר עותק קל של המודל ("תאום דיגיטלי"), ובו רק התיבה וקו המרכז של כל אלמנט. העותק מוצא אילו תעלות הקורה חוסמת עכשיו ומחשב להן מסלול חדש תוך פחות מאלפית שנייה. אחר כך הוא בודק את המסלול ושולח למתכנן מערכות המיזוג (מהנדס המכונות) **הצעה**. ההצעה מוצגת כשכבת תצוגה זמנית מעל המודל שלו: רואים אותה, אבל היא לא חלק מהמודל. שום דבר לא משתנה עד שהוא לוחץ "אשר". גם האדריכל מקבל הודעה: "ההזזה שלך משפיעה על 3 תעלות". זו ההתראה הכי שימושית במוצר.

למה זה בתוך שניות ולא תוקע את Revit

- שולחים הבדלים, לא מודלים. המודל הכבד לא עוזב את המחשב של המשתמש.
- ההצעה היא שכבת תצוגה, לא עריכה. Revit לא מחשב מחדש כלום עד האישור.
- שינוי אחד שמשפיע על 200 תעלות נשלח כהודעה אחת, לא 200.
- יעד: חצי שנייה בחצי מהמקרים, ועד 3 שניות ב-99% מהמקרים. מה שלא בשליטתנו: הזמן ש-Revit עצמו לוקח לעדכן את המודל אחרי השינוי.

הפרטים שמי שמכיר Revit יחפש

- הקורה של הקונסטרוקטור יושבת במודל מקושר (קובץ נפרד שמוצג בתוך המודל של מתכנן המיזוג) ומתעדכנת אצלו רק אחרי "reload". הענן מגשר על הפער.
- כל אלמנט מזוהה לפי `UniqueId` (מזהה קבוע שנותן Revit), והגרסה שלו לפי `VersionGuid` (מזהה שמתחלף בכל שינוי). היחידות מומרות מרגל (היחידה הפנימית של Revit) למטר.
- אישור הצעה בודק שאף אחד מהאלמנטים שהיא תלויה בהם לא השתנה בינתיים. אם אחד מהם השתנה, ההצעה נפסלת ומבקשים הצעה חדשה.
- התקשורת עוברת ב-WebSocket (חיבור פתוח וקבוע בין המחשב לשרת), שמתאים גם לתוספי Revit ב-.NET Framework וגם לדפדפן. ההודעות עוברות בתור ששומר על הסדר, ואחרי ניתוק החיבור ממשיך מהנקודה שבה נעצר.

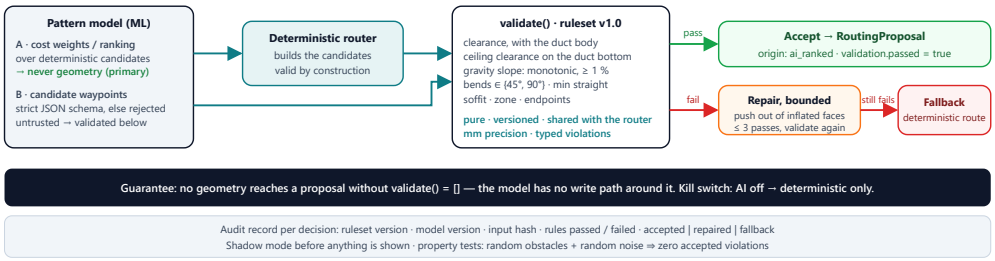


מסלול המידע: התוסף ב-Revit ← שרת כניסה (Gateway) ← תור הודעות ← מנוע הניתוב ← שליחה למתכנן המיזוג, לדף הצפייה בדפדפן ולאדריכל.

3 שילוב AI בצורה בטוחה, וחלוקת העבודה מול המפתחת של התוסף

העיקרון. ה-AI רשאי **להמליץ ולדרג** בין מסלולים שהמנוע הדטרמיניסטי (קוד עם כללים קבועים: אותו קלט נותן תמיד אותה תוצאה) בנה, אבל הוא **לא קובע בעצמו נקודות של מסלול**. כך כל מה שהוא יכול לבחור הוא מסלול שהמנוע כבר בנה ובדק. אם בכל זאת נותנים למודל ה-AI להציע נקודות, ההצעה עוברת את אותם כללים קשיחים: מרווח מהמכשולים, תחתית התעלה מעל התקרה המונמכת, שיפוע לצנרת שזורמת בכוח הכובד, כמו ניקוז וביוב (לפחות 1%, ואסור לה לעלות), זוויות של 45° או 90° בלבד, אורך ישר מינימלי בין אביזרים. נכשלה? מנסים תיקון אוטומטי מוגבל (עד שלושה סבבים): דוחפים את הנקודות אל מחוץ לתיבה המוגדלת, ובודקים שוב. עדיין נכשלה? חוזרים למסלול הדטרמיניסטי כגיבוי. **כל החלטה נרשמת**: איזו גרסת כללים, איזו גרסה של מודל ה-AI, אילו כללים עברו ונכשלו. מריצים קודם ב"מצב צל" (המודל רץ ונרשם, אבל ההצעות שלו לא מוצגות), ויש מתג כיבוי.

החוזה מול המפתחת של תוסף ה-Revit. שישה סוגי הודעות בפורמט JSON (פורמט טקסט סטנדרטי להעברת נתונים), ולכל אחד סכמה (הגדרה מדויקת של השדות) ודוגמה בריפו: שינוי במודל (מהתוסף למנוע), בקשת ניתוב, תשובת ניתוב, הצעת ניתוב (מהמנוע ללקוחות), החלטה (אישור או דחייה), ושגיאה. חלוקת העבודה ברורה: היא אחראית ללכידת השינויים ב-Revit, להמרת היחידות, לשכבת התצוגה ולהפיכת הצעה מאושרת לתעלות ואביזרים אמיתיים; המנוע, הבדיקות ושליחת ההצעות למשתמשים אצלי. **קובצי הדוגמה משמשים "מנוע מדומה"**: התוסף מקבל מהם תשובות קבועות, כאילו הגיעו מהמנוע האמיתי. כך היא יכולה לבנות ולהדגים את התוסף עוד לפני שהמנוע קיים.



שער הבטיחות: מודל ה-AI ממליץ, הבודק הדטרמיניסטי מחליט, תיקון מוגבל, חזרה למסלול הדטרמיניסטי כגיבוי, ורישום של כל החלטה.

· איך לעבור על החומר בעשר דקות

1. לפתוח את הדף maof-mep.vercel.app, להזיז את הקורה בסליידר ולראות את התעלה מנותבת מחדש; לבחור תעלה 300×400 ולהדליק את התקרה כדי לראות "אין מסלול" עם סיבה.
2. לקרוא את שני עמודי המסמך באנגלית: שם ההנמקה המלאה, הפסאודו-קוד, ה-JSON והתרשימים.
3. בריפו: `python -m routing` מדפיס את המספרים, `python -m pytest` מריץ 22 טסטים, ובתיקייה `schema/` הסכמות והדוגמאות.

אלנתן דוידוביץ · 050-6998223 · elnatan1990@gmail.com · כל המספרים במסמך מחושבים על ידי הקוד ומאומתים בטסטים, לא נכתבו ביד.